

pfl-ufg / **problema-6---subprogramas-Merus23** Private[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#) [Security](#) [Insights](#) master**problema-6---subprogramas-Merus23** / problema-6-subprogramas.md

rcarocho feedback do professor



3 contributors



# Problema 6: Subprogramas

## Identificação

- Aluno: "Mateus Pereira da Silva "

## Objetivos

- Verificar a compreensão do conceito de implementação de subprogramas.

## Formato das respostas

Todas as suas respostas devem ser inseridas neste documento, utilizando o formato Markdown. Exceto quando informando explicitamente, as respostas devem ser escritas utilizando a seguinte formatação:

- Código: formato de código do Markdown Github
- Texto livre: texto normal com parágrafos.
- Tabela: Tabela markdown seguindo o formato abaixo:
  - Documentação do formato de tabelas no Markdown Github:  
<https://docs.github.com/en/github/writing-on-github/working-with-advanced-formatting/organizing-information-with-tables>

Sempre que houver um conteúdo externo, como figuras ou código, ele deverá ser colocado no diretório e arquivos indicados no exemplo de resposta.

Na seção "**Feedback**" ao fim deste documento, o professor incluirá a avaliação da sua resposta.

## Critério de Avaliação

---

A atividade é **individual**. Qualquer cópia fará a atividade ser desconsiderada (nota zero).

## Questão 1

---

Considere duas implementações do algoritmo de ordenação **Bubble Sort**, descritas nos arquivos `bsort-iterativo.c` e `bsort-recursivo.c`, referentes à implementação iterativa e recursiva do algoritmo, respectivamente.

Descreva em um diagrama (tabela) a configuração em memória da pilha dinâmica de chamadas de funções (incluindo ARI), quando a pilha estiver com seu **maior tamanho**, ou maior número de chamadas encadeadas, quando o algoritmo de ordenação tiver que ordenar o vetor  $\{ 8, 3, 5, 1 \}$ .

Para descrever o diagrama, utilize uma tabela markdown seguindo a estrutura a seguir:

| Posição memória | Valor | Significado                      |
|-----------------|-------|----------------------------------|
| 00000           | 2     | ponteiro para <code>x</code>     |
| 00001           | 23    | valor da variável <code>j</code> |
| 00002           | 100   | valor da variável <code>x</code> |

A coluna "Posição memória" é o endereço de memória onde o dado da pilha está armazenado: **não é a posição na pilha** e uma nova entrada na pilha sempre deverá ficar na **próxima** posição de memória disponível (sempre crescente). A coluna "*Valor*" contém o valor armazenado na posição e a coluna "*Significado*" é apenas uma descrição para ajudar a interpretação daquela entrada e a compreender a tabela (não faz parte da memória). Todos os valores armazenados são decimais e todas as posições de memória são suficientes para armazenar tanto endereços de memória como inteiros usados no código.

Como a pilha precisará ter ponteiros para os valores de retorno no código, considere os seguintes endereço de memória em relação aos códigos:

Para implementação `bsort-recursivo.c`

| Função     | Instrução               | Linha do código | Posição na memória |
|------------|-------------------------|-----------------|--------------------|
| troca      | primeira                | 7               | 157                |
| bubbleSort | primeira                | 14              | 164                |
| bubbleSort | após chamada troca      | 19              | 169                |
| bubbleSort | após chamada bubbleSort | 24              | 174                |
| main       | primeira                | 27              | 177                |
| main       | após chamada bubbleSort | 31              | 181                |

Para implementação `bsort-iterativo.c`

| Função     | Instrução               | Linha do código | Posição na memória |
|------------|-------------------------|-----------------|--------------------|
| troca      | primeira                | 7               | 257                |
| bubbleSort | primeira                | 15              | 265                |
| bubbleSort | após chamada troca      | 21              | 271                |
| main       | primeira                | 27              | 277                |
| main       | após chamada bubbleSort | 31              | 181                |

Alguns aspectos adicionais:

- Considere as variáveis locais de cada função, mas apenas aquelas explicitamente declaradas como `k` de `BubbleSort`.
- Não esqueça de colocar a invocação à função `main()`.

**IMPORTANTE:** Sugestões que irão auxiliar na resolução da questão:

- Execute o código no cenário indicado, colocando `printf` nas invocações, para ter certeza que está considerando as invocações certas das funções. Se você errar nas invocações, naturalmente a sua pilha estará errada.
- Na sua entrada na tabela coloque quantas entradas forem necessárias. Leia e acompanhe este enunciado na **interface web do Github** e quando estiver escrevendo a sua resposta, verifique se a formatação da sua resposta está correta e adequada, usando também a interface web.

- Não deixe para fazer de última hora e tire as dúvidas com o professor usando o Google Classroom.

### Resposta: Configuração Máxima da Pilha em Memória

Para implementação `bsort-recursivo.c`

Altere a tabela abaixo de acordo com a sua resposta

| Posição memória | Valor | Significado                                                  |
|-----------------|-------|--------------------------------------------------------------|
| 00000           | 8     | variável <code>arr[0]</code> no main                         |
| 00001           | 3     | variável <code>arr[1]</code> no main                         |
| 00002           | 5     | variável <code>arr[2]</code> no main                         |
| 00003           | 1     | variável <code>arr[3]</code> no main                         |
| 00004           | 4     | variável <code>n</code> no main                              |
| 00005           | 0     | variável local <code>i</code> no bubbleSort primeira chamada |
| 00005           | 1     | variável local <code>i</code> no bubbleSort primeira chamada |
| 00005           | 2     | variável local <code>i</code> no bubbleSort primeira chamada |
| 00006           | 3     | parâmetro <code>arr[0]</code> do bubbleSort primeira chamada |
| 00007           | 5     | parâmetro <code>arr[1]</code> do bubbleSort primeira chamada |
| 00008           | 1     | parâmetro <code>arr[2]</code> do bubbleSort primeira chamada |
| 00009           | 8     | parâmetro <code>arr[3]</code> do bubbleSort primeira chamada |
| 000010          | 4     | parâmetro <code>n</code> do bubbleSort primeira chamada      |
| 000011          | 0     | variável local <code>i</code> no bubbleSort segunda chamada  |
| 000011          | 1     | variável local <code>i</code> no bubbleSort segunda chamada  |
| 000011          | 2     | variável local <code>i</code> no bubbleSort segunda chamada  |
| 00006           | 3     | parâmetro <code>arr[0]</code> do bubbleSort segunda chamada  |

| Posição memória | Valor | Significado                                     |
|-----------------|-------|-------------------------------------------------|
| 00007           | 1     | parâmetro arr[1] do bubbleSort segunda chamada  |
| 00008           | 5     | parâmetro arr[2] do bubbleSort segunda chamada  |
| 00009           | 8     | parâmetro arr[3] do bubbleSort segunda chamada  |
| 000010          | 3     | parâmetro n do bubbleSort segunda chamada       |
| 000011          | 0     | variável local i no bubbleSort terceira chamada |
| 000011          | 1     | variável local i no bubbleSort terceira chamada |
| 000011          | 2     | variável local i no bubbleSort terceira chamada |
| 000012          | 1     | parâmetro arr[0] do bubbleSort terceira chamada |
| 000013          | 3     | parâmetro arr[1] do bubbleSort terceira chamada |
| 000014          | 5     | parâmetro arr[2] do bubbleSort terceira chamada |
| 000015          | 8     | parâmetro arr[3] do bubbleSort terceira chamada |
| 000016          | 2     | parâmetro n do bubbleSort terceira chamada      |
| 000017          | 8     | parâmetro arr[0] da troca                       |
| 000018          | 3     | parâmetro arr[1] da troca                       |
| 000019          | 5     | parâmetro arr[2] da troca                       |
| 000020          | 1     | parâmetro arr[3] da troca                       |
| 000017          | 3     | parâmetro arr[0] da troca                       |
| 000018          | 8     | parâmetro arr[1] da troca                       |
| 000019          | 5     | parâmetro arr[2] da troca                       |
| 000020          | 1     | parâmetro arr[3] da troca                       |
| 000017          | 3     | parâmetro arr[0] da troca                       |
| 000018          | 5     | parâmetro arr[1] da troca                       |

| Posição memória | Valor | Significado               |
|-----------------|-------|---------------------------|
| 000019          | 8     | parâmetro arr[2] da troca |
| 000020          | 1     | parâmetro arr[3] da troca |
| 000017          | 3     | parâmetro arr[0] da troca |
| 000018          | 5     | parâmetro arr[1] da troca |
| 000019          | 1     | parâmetro arr[2] da troca |
| 000020          | 8     | parâmetro arr[3] da troca |
| 000017          | 3     | parâmetro arr[0] da troca |
| 000018          | 1     | parâmetro arr[1] da troca |
| 000019          | 5     | parâmetro arr[2] da troca |
| 000020          | 8     | parâmetro arr[3] da troca |
| 000017          | 1     | parâmetro arr[0] da troca |
| 000018          | 3     | parâmetro arr[1] da troca |
| 000019          | 5     | parâmetro arr[2] da troca |
| 000020          | 8     | parâmetro arr[3] da troca |
| 000021          | 0     | parâmetro i da troca      |
| 000022          | 1     | parâmetro j da troca      |
| 000021          | 1     | parâmetro i da troca      |
| 000022          | 2     | parâmetro j da troca      |
| 000021          | 2     | parâmetro i da troca      |
| 000022          | 3     | parâmetro j da troca      |
| 000021          | 1     | parâmetro i da troca      |
| 000022          | 2     | parâmetro j da troca      |
| 000021          | 0     | parâmetro i da troca      |
| 000022          | 1     | parâmetro j da troca      |

### Para implementação `bsort-iterativo.c`

Altere a tabela abaixo de acordo com a sua resposta

| Posição memória | Valor | Significado                    |
|-----------------|-------|--------------------------------|
| 00000           | 8     | variável local arr[0] no main  |
| 00001           | 3     | variável local arr[1] no main  |
| 00002           | 5     | variável local arr[2] no main  |
| 00003           | 1     | variável local arr[3] no main  |
| 00004           | 4     | variável local n no main       |
| 00005           | 0     | variável local k no bubbleSort |
| 00005           | 1     | variável local k no bubbleSort |
| 00005           | 2     | variável local k no bubbleSort |
| 00006           | 0     | variável local i no bubbleSort |
| 00006           | 1     | variável local i no bubbleSort |
| 00006           | 2     | variável local i no bubbleSort |
| 00006           | 0     | variável local i no bubbleSort |
| 00006           | 1     | variável local i no bubbleSort |
| 00006           | 0     | variável local i no bubbleSort |
| 00007           | 8     | parâmetro arr[0] do bubbleSort |
| 00008           | 3     | parâmetro arr[1] do bubbleSort |
| 00009           | 5     | parâmetro arr[2] do bubbleSort |
| 000010          | 1     | parâmetro arr[3] do bubbleSort |
| 000011          | 4     | parâmetro n do bubbleSort      |
| 000012          | 8     | variável local temp no troca   |
| 000012          | 8     | variável local temp no troca   |
| 000012          | 8     | variável local temp no troca   |
| 000012          | 5     | variável local temp no troca   |
| 000012          | 3     | variável local temp no troca   |
| 000013          | 8     | parâmetro arr[0] da troca      |
| 000014          | 3     | parâmetro arr[1] da troca      |

| Posição memória | Valor | Significado               |
|-----------------|-------|---------------------------|
| 000016          | 1     | parâmetro arr[3] da troca |
| 000013          | 3     | parâmetro arr[0] da troca |
| 000014          | 8     | parâmetro arr[1] da troca |
| 000015          | 5     | parâmetro arr[2] da troca |
| 000016          | 1     | parâmetro arr[3] da troca |
| 000013          | 3     | parâmetro arr[0] da troca |
| 000014          | 5     | parâmetro arr[1] da troca |
| 000015          | 8     | parâmetro arr[2] da troca |
| 000016          | 1     | parâmetro arr[3] da troca |
| 000013          | 3     | parâmetro arr[0] da troca |
| 000014          | 5     | parâmetro arr[1] da troca |
| 000015          | 1     | parâmetro arr[2] da troca |
| 000016          | 8     | parâmetro arr[3] da troca |
| 000013          | 3     | parâmetro arr[0] da troca |
| 000014          | 1     | parâmetro arr[1] da troca |
| 000015          | 5     | parâmetro arr[2] da troca |
| 000016          | 8     | parâmetro arr[3] da troca |
| 000013          | 1     | parâmetro arr[0] da troca |
| 000014          | 3     | parâmetro arr[1] da troca |
| 000015          | 5     | parâmetro arr[2] da troca |
| 000016          | 8     | parâmetro arr[3] da troca |
| 000017          | 0     | parâmetro i da troca      |
| 000018          | 1     | parâmetro j da troca      |
| 000017          | 1     | parâmetro i da troca      |
| 000018          | 2     | parâmetro j da troca      |
| 000017          | 2     | parâmetro i da troca      |
| 000018          | 3     | parâmetro j da troca      |



| Posição memória | Valor | Significado                 |
|-----------------|-------|-----------------------------|
| 000017          | 1     | parâmetro <i>i</i> da troca |
| 000018          | 2     | parâmetro <i>j</i> da troca |
| 000017          | 0     | parâmetro <i>i</i> da troca |
| 000018          | 1     | parâmetro <i>j</i> da troca |

### Tamanho em bytes da Pilha

Qual o tamanho máximo em bytes da pilha dinâmica nas duas configurações, considerando que o inteiro usado ocupa 4 bytes.

- Para implementação `bsort-recursivo.c` :  $22 * 4 = 88$  bytes
- Para implementação `bsort-iterativo.c` :  $18 * 4 = 72$  bytes

## Feedback do Professor

*Esta seção será escrita pelo professor ao final da avaliação da sua tarefa.*

- 'MATEUS PEREIRA DA SILVA' ==> 10,0 (problema Problema.6.subprogramas)
- **Revisões/Problemas:**
  - A sua atividade foi submetida com **atraso de 1 dias**.
  - **Q1.1-recur**: Ok (não foi verificada em detalhes)
  - **Q1.2-itera**: Ok (não foi verificada em detalhes)
  - **Q1.3-taman**: Ok (não foi verificada em detalhes)